

Clase 03 - Operaciones aritméticas lógicas y almacenamiento de datos

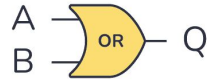
Profesor: **IIC2343 - Arquitectura de Computadores**
- Felipe Valenzuela González
Correo:
frvalenzuela@alumni.uc.cl

Resumen de la clase pasada

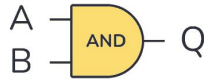
Compuertas lógicas



A	Q
0	1
1	0



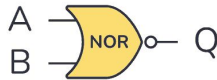
A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1



A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0



A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0



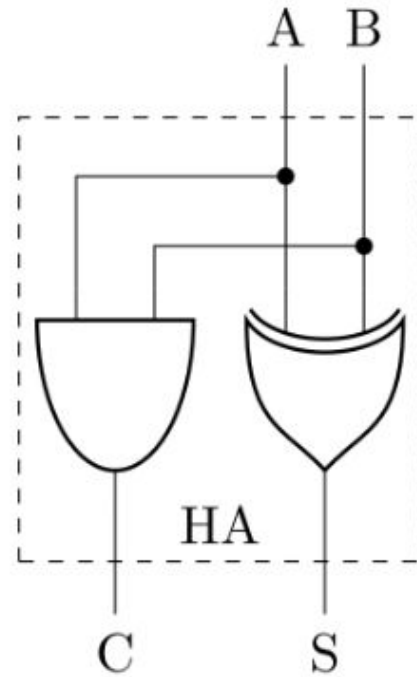
A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0



A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1

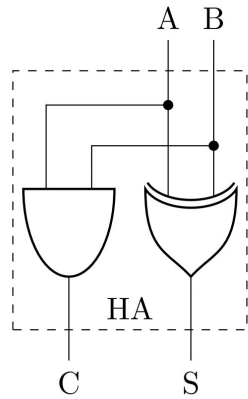
Half Adder

<i>A</i>	<i>B</i>	<i>S</i>	<i>C</i>
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

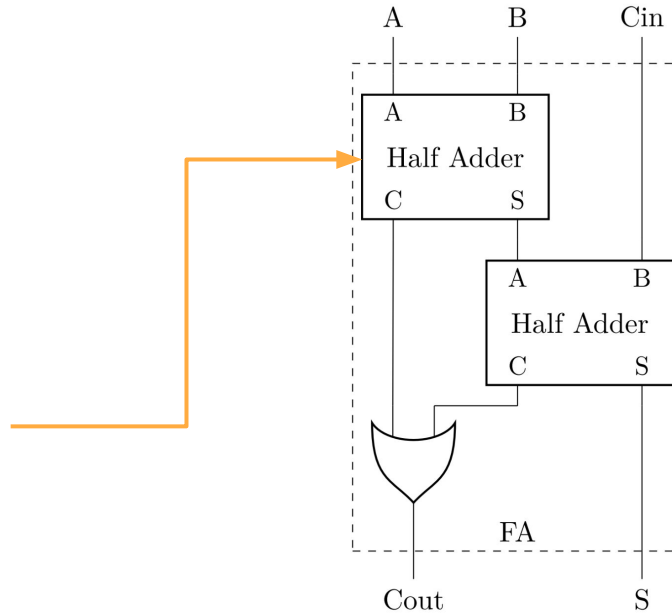


(a) Half Adder

Full Adder

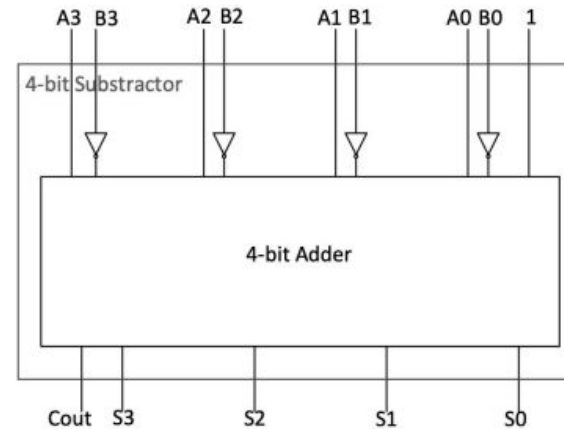
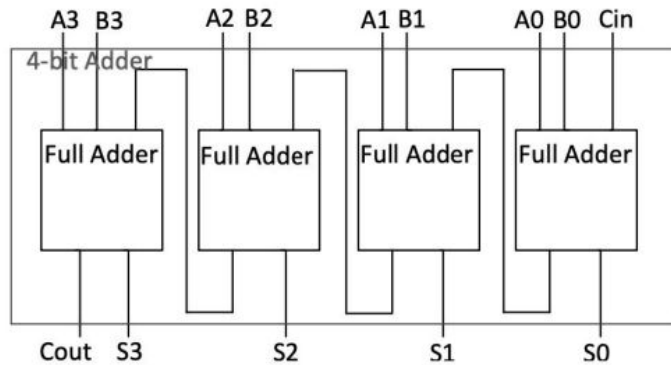


(a) Half Adder



(b) Full Adder

Sumador/Restador



¿Dudas?

Introducción:

- Un computador lo definimos como una **máquina programable que ejecuta programas.**
- Para programar necesitamos:
 - Datos: números (enteros, reales) , texto, imágenes, etc
 - Operaciones: suma, resta, multiplicación, división, etc
 - Variables: simples, arreglos
 - Control de flujo: comparaciones, manejo de ciclos
- La clase de hoy veremos más **operaciones, y lo básico de variables y control**



Objetivos de la clase

- Conocer más operaciones aritméticas y lógicas
- Conocer circuitos básicos de control de flujo
- Entendimiento de almacenamiento de datos

Resolución de circuitos

Minterm

- Las señales de **input** se conectan con compuertas **AND**
- Los valores 0 se niegan con la compuerta **NOT**
- **Todas las combinaciones** se conectan con **compuertas OR**
- Las combinación de de **input** debe dar 1

A	B	C	Output
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Minterm

Ejemplo:

NOT(A) AND B AND NOT(C)

OR

A AND NOT(B) AND C

OR

A AND B AND C

A	B	C	Output
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Maxterm

- Las señales de **input** se conectan con compuertas **OR**
- Los valores 1 se niegan con la compuerta **NOT**
- **Todas las combinaciones** se conectan con **compuertas OR**
- Las combinación de de **input** debe dar 0

A	B	C	Output
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Maxterm

Ejemplo:

A OR B OR C

AND A OR B OR NOT(C)

AND A OR NOT(B) OR NOT(C)

AND NOT(A) OR B OR C

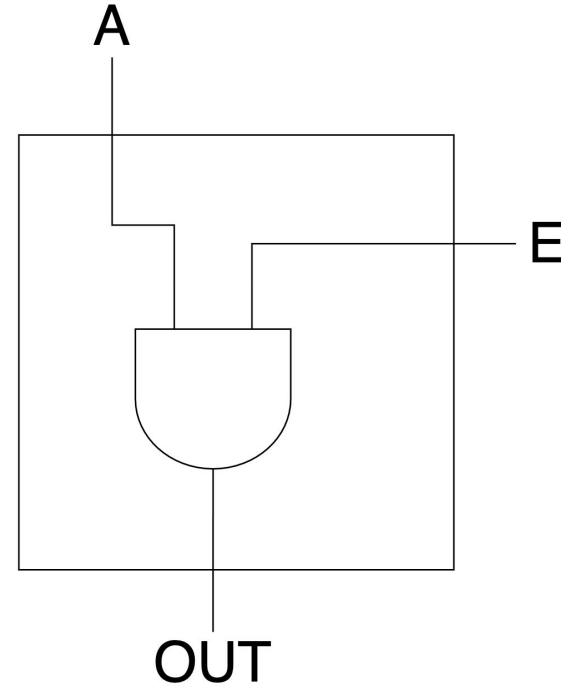
AND NOT(A) OR NOT(B) OR C

A	B	C	Output
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

¿Dudas?

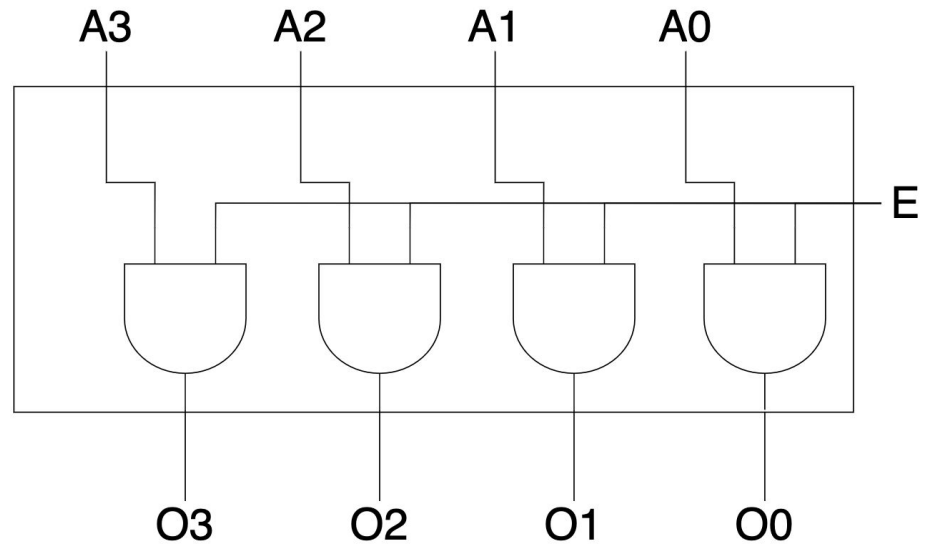
Circuitos de control - Enabler

OUT	E
0	0
A	1



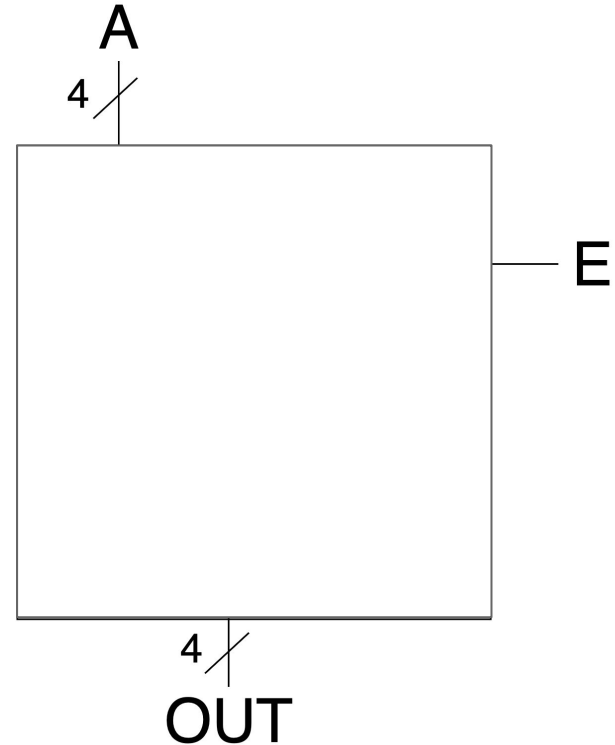
Circuitos de control - Enabler 4 bits

O3O2O1O0	E
0	0
A3A2A1A0	1



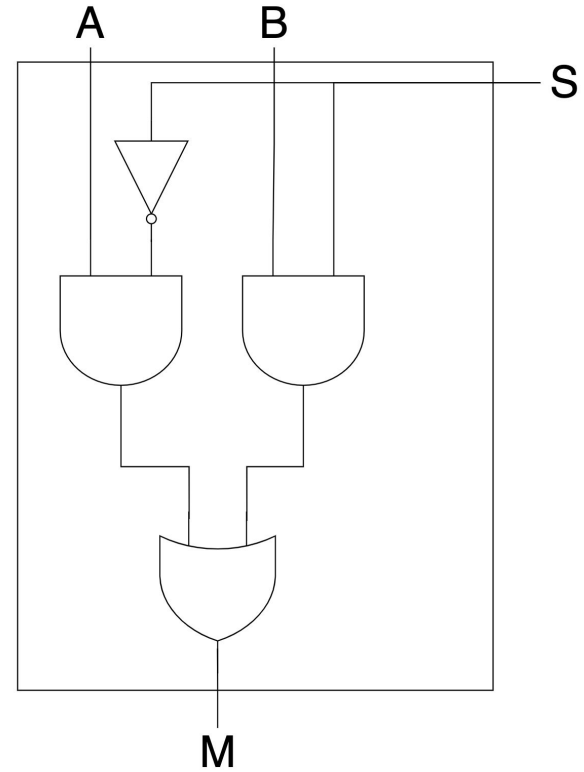
Circuitos de control - Enabler - Bus Format

OUT	E
0	0
A	1



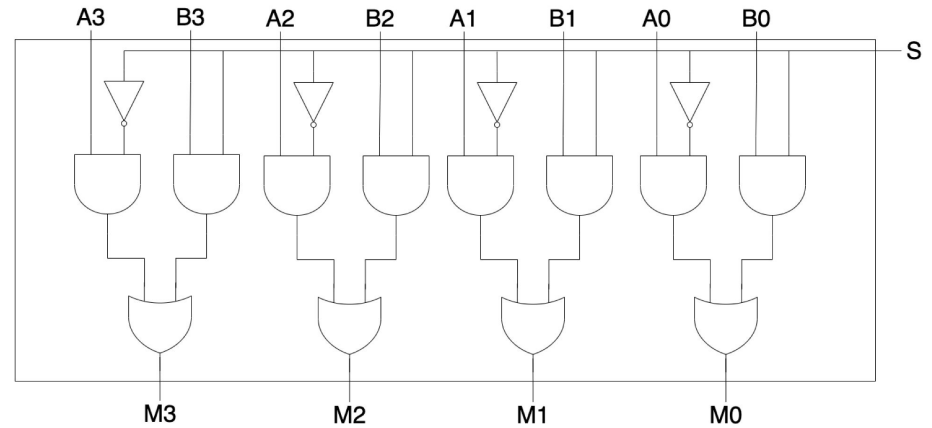
Circuitos de control - Multiplexor

S	M
0	A
1	B



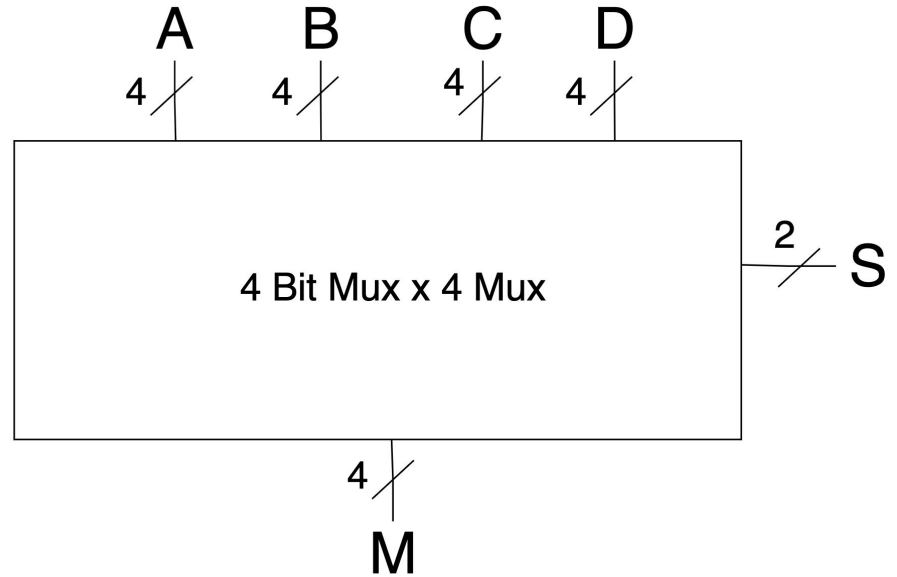
Circuitos de control - Multiplexor 4 bits

S	M
0	A
1	B



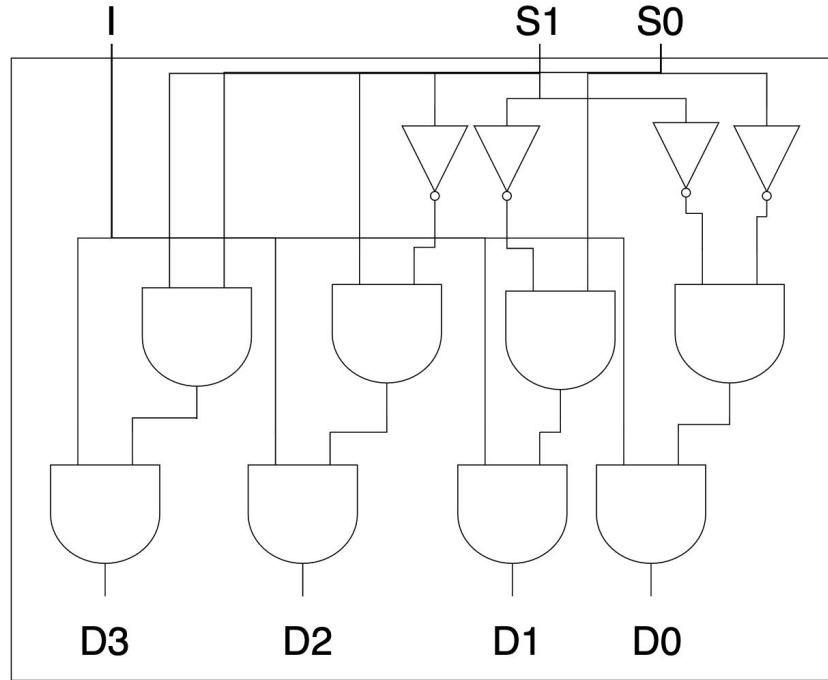
Circuitos de control - 4 bit x 4 Mux

S1	S0	M
0	0	A
0	1	B
1	0	C
1	1	D



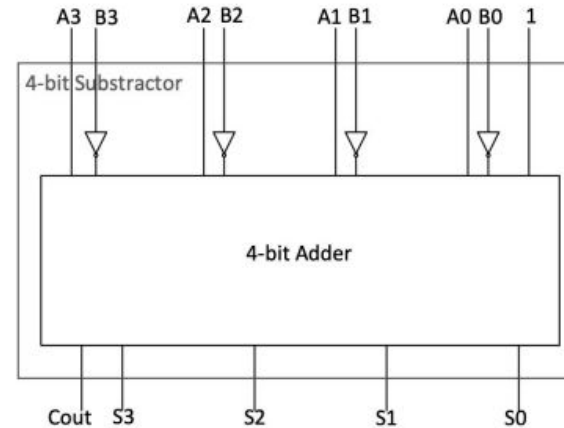
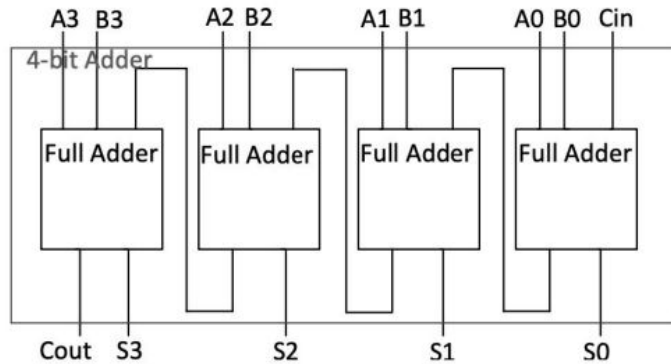
Circuitos de control - Demux

S1	S0	Output
0	0	D0 = I
0	1	D1 = I
1	0	D2 = I
1	1	D3 = I



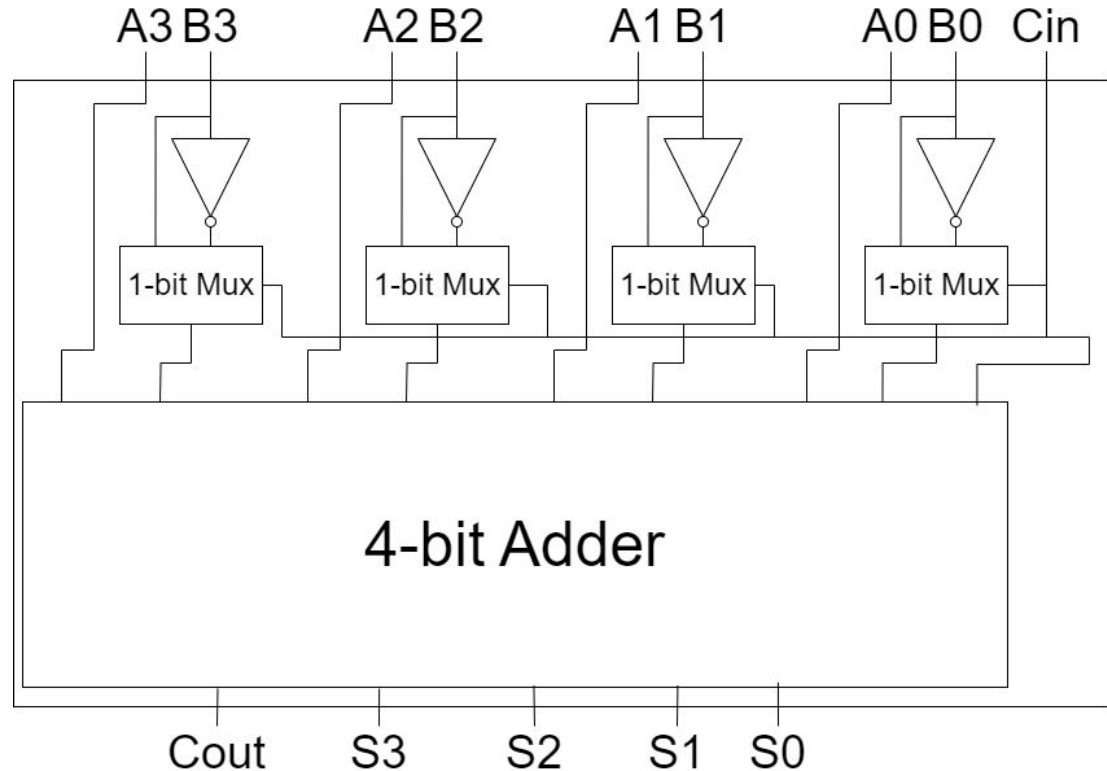
¿Dudas?

Operaciones: Sumador y Restador de 4 bits



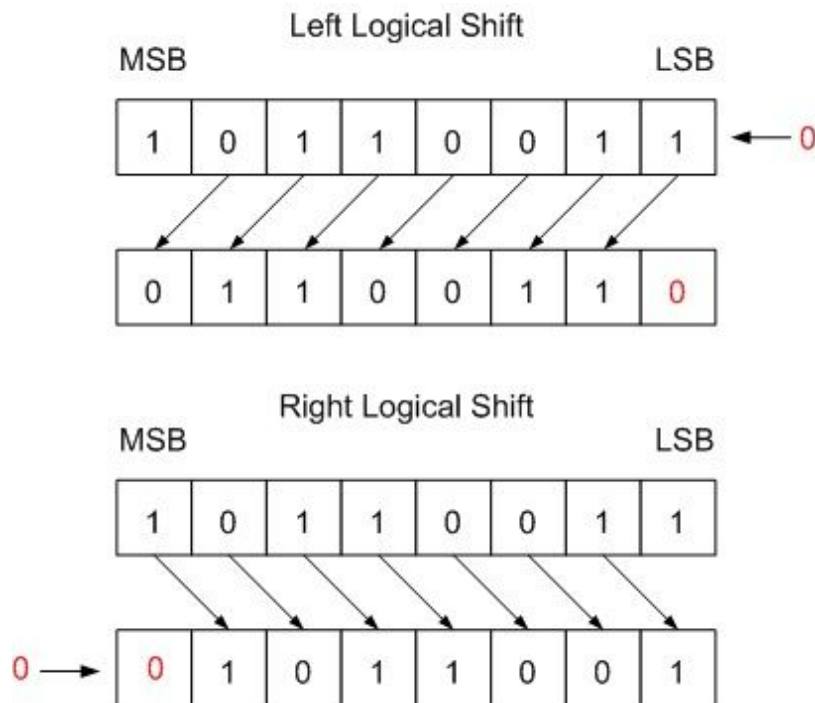
Operaciones: Sumador-Restador de 4 Bits

Cin	Result
0	$A+B$
1	$A-B$



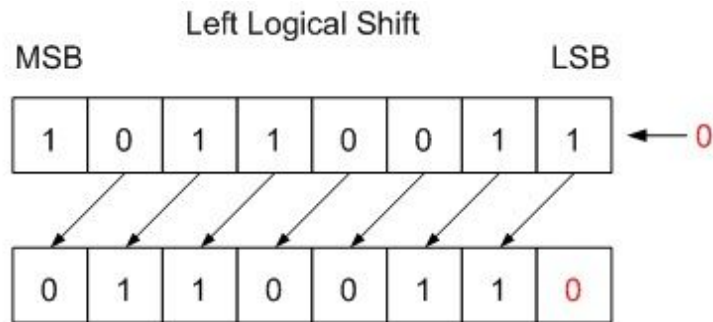
Operaciones: Lógicas - Shift Lógico

- Una operación muy ocupada es el **shifting** o desplazamiento.
- Consiste en mover todas las cifras a la izquierda o la derecha.



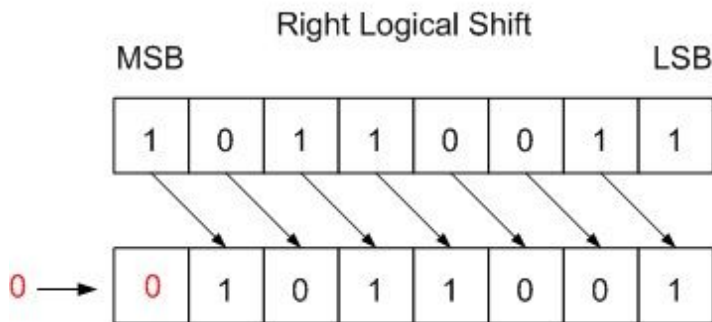
Operaciones: Lógicas - Shift Left Lógico (SHL)

- Definiremos el desplazamiento a la izquierda como **SHIFT LEFT (SHL)**
- En binario, desplazar a la izquierda es análogo a multiplicar por 2
- Se agrega un cero en el bit **menos** significativo



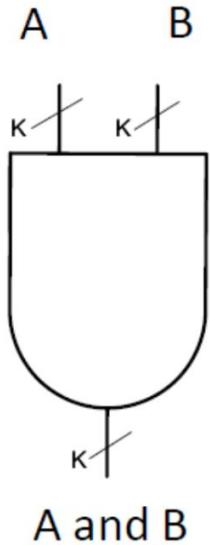
Operaciones: Lógicas - Shift Right Lógico (SHR)

- Definiremos el desplazamiento a la izquierda como **SHIFT RIGHT (SHR)**
- En binario, desplazar a la izquierda es análogo a **división entera por 2**
- Se agrega un cero en el bit **más** significativo



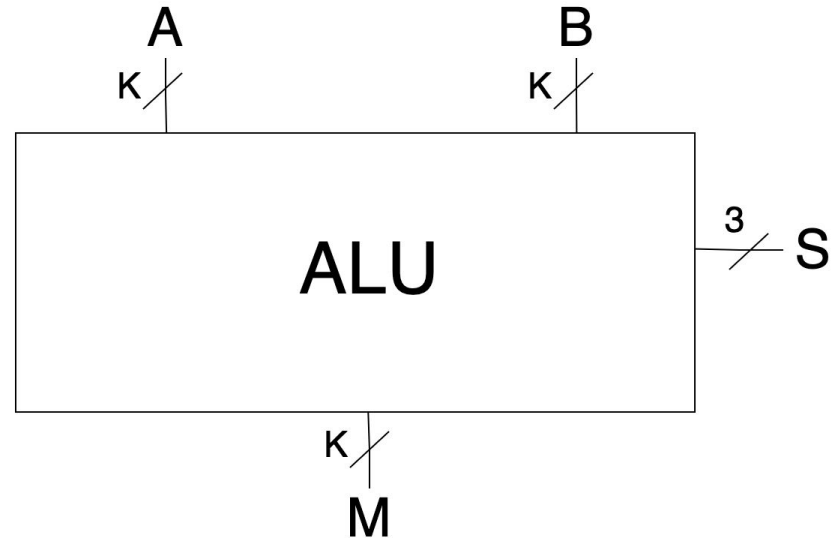
¿Dudas?

Operaciones: Operadores lógicos *bitwise*



Unidad Aritmética Lógica: ALU

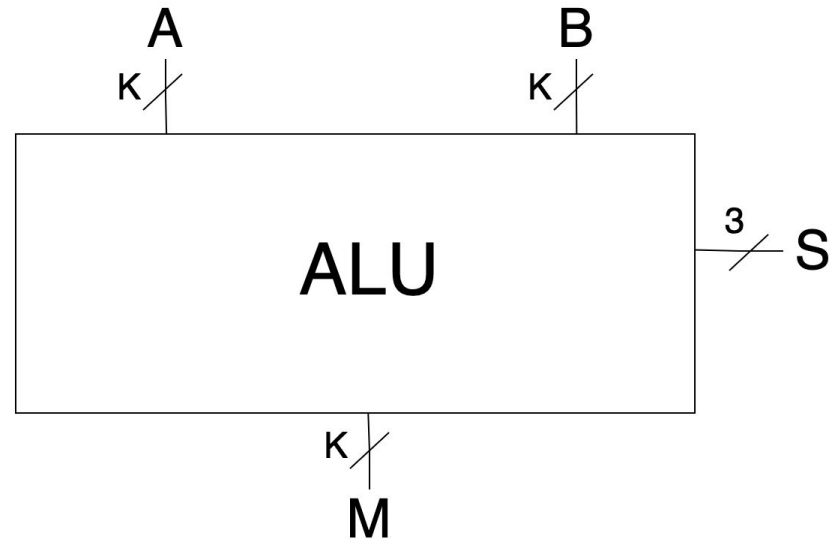
- Es el **componente principal** para hacer operaciones en un computador
- Se basa en **operaciones lógicas *bitwise*** y operaciones aritméticas como lo son el **sumador-restador**
- Su comportamiento interno lo verán **próxima semana** en la actividad de laboratorio



Unidad Aritmética Lógica: ALU

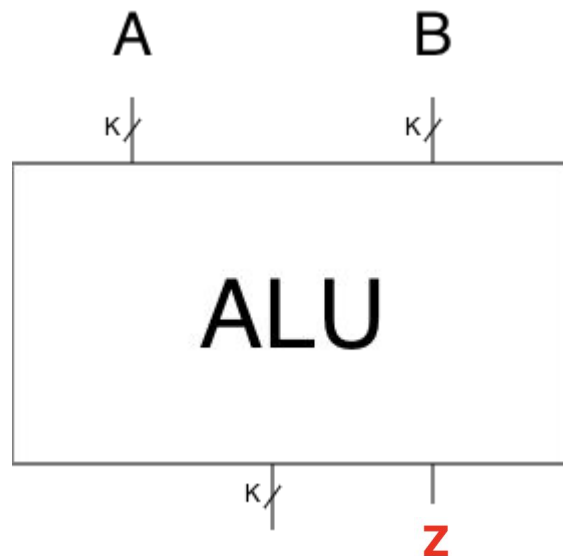
- Tabla de valores:

S2	S1	S0	M
0	0	0	Suma
0	0	1	Resta
0	1	0	And
0	1	1	Or
1	0	0	Not
1	0	1	Xor
1	1	0	Shift left
1	1	1	Shift right



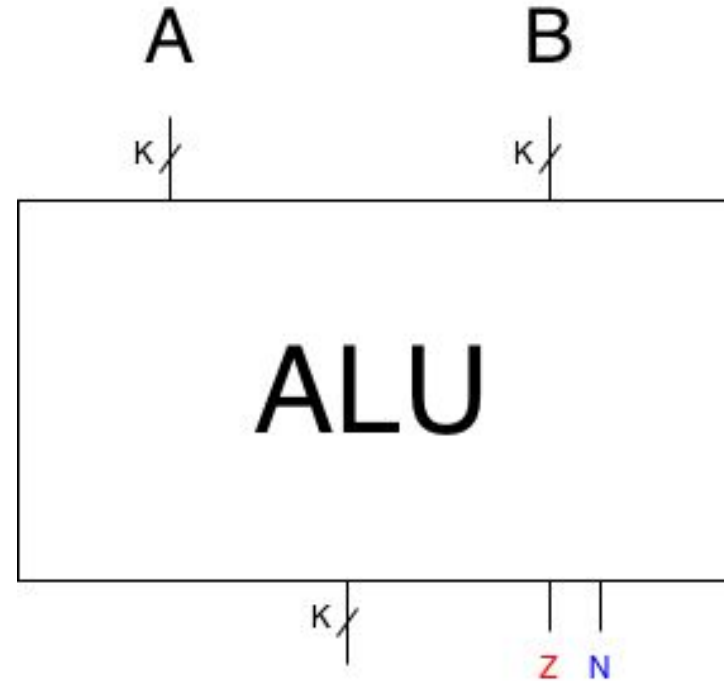
Unidad Aritmética Lógica: *Flags*

- De la ALU ahora obtenemos flags (señales de 1 bit) que nos indican el cumplimiento o no de una condición. Particularmente:
- **Z: Zero**. Indica si el resultado de la operación fue cero o no.



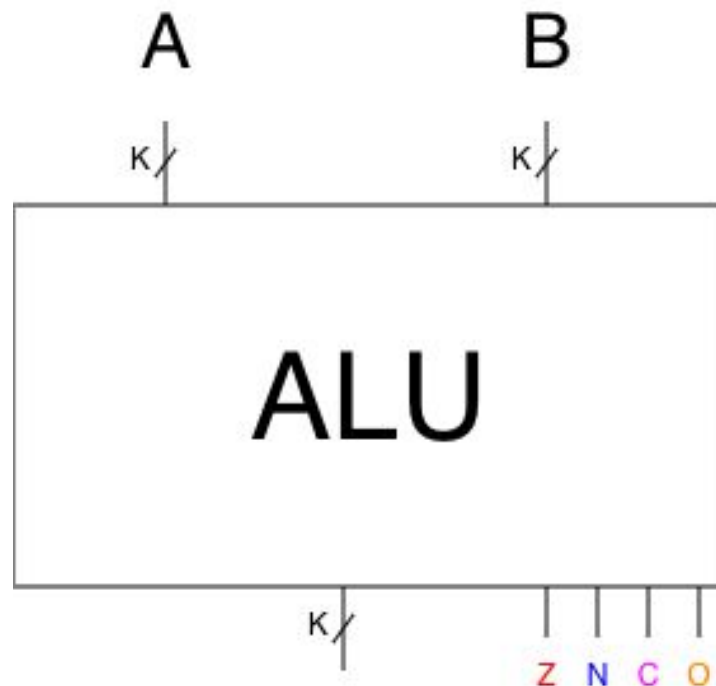
Unidad Aritmética Lógica: *Flags*

- De la ALU ahora obtenemos flags (señales de 1 bit) que nos indican el cumplimiento o no de una condición. Particularmente:
- **Z: Zero**. Indica si el resultado de la operación fue cero o no.
- **N: Negative** Indica si el resultado de la operación fue negativo o no



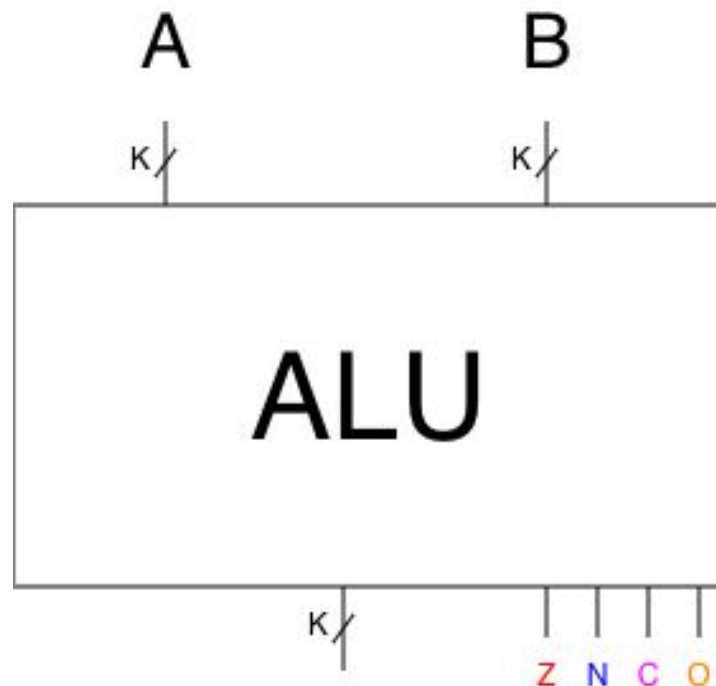
Unidad Aritmética Lógica: *Flags*

- **C: Carry**. Indica si hubo un bit de carry al finalizar la operación.
- **O: Overflow**. Indica si hubo o no overflow al finalizar la operación.
- Ahondaremos un poco más en estas nuevas flags.



Unidad Aritmética Lógica: *Flags* - C y O

- **Flag C:** Es 1 si hubo bit de carry en una operación ADD/SUB, pero también se utiliza para representar el bit “descartado” en operaciones tipo shift.
 - Ejemplo: SHR(00101011) → C = 1
- **Flag O:** Es 1 si hubo overflow en operaciones ADD/SUB. Algunas implementaciones también lo consideran en operaciones tipo shift, pero no todas.
- Existen cuatro casos a considerar:
 - $A \geq 0; B \geq 0 \rightarrow O = 1$ si $A + B < 0$
 - $A < 0; B < 0 \rightarrow O = 1$ si $A + B \geq 0$
 - $A \geq 0; B < 0 \rightarrow O = 1$ si $A - B < 0$
 - $A < 0; B \geq 0 \rightarrow O = 1$ si $A - B \geq 0$



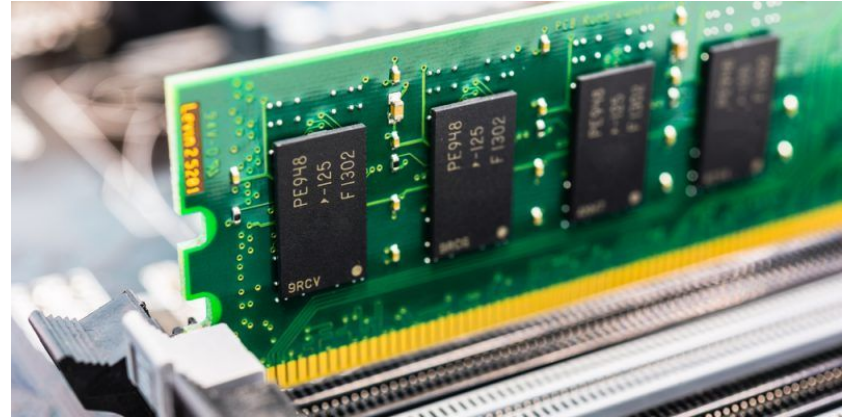
Introducción del curso:

- Un computador lo definimos como una **máquina programable que ejecuta programas.**
- Para programar necesitamos:
 - Datos: números (enteros, reales) , texto, imágenes, etc
 - Operaciones: suma, resta, multiplicación, división, etc
 - Variables: simples, arreglos
 - Control de flujo: comparaciones, manejo de ciclos
- Para continuar la clase de hoy, y veremos **lo básico para tener variables**



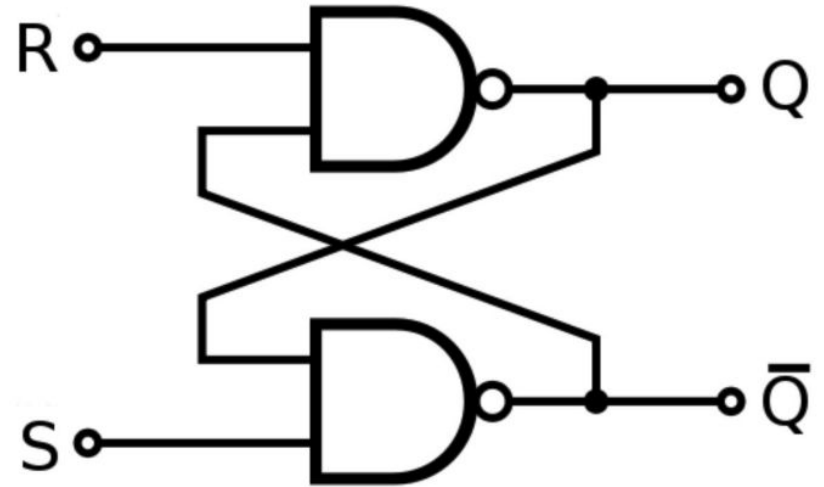
Almacenamiento de datos

- Es componente esencial de todo computador
- Es *hardware* especializado para **almacenar** un estado
- Permite realizar **un cambio** en un **instante determinado de tiempo**



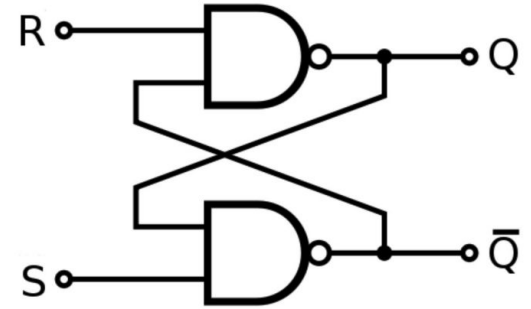
Almacenamiento de datos: Latch RS

S	R	Q(t+1)
0	0	-
0	1	0
1	0	1
1	1	Q(t)



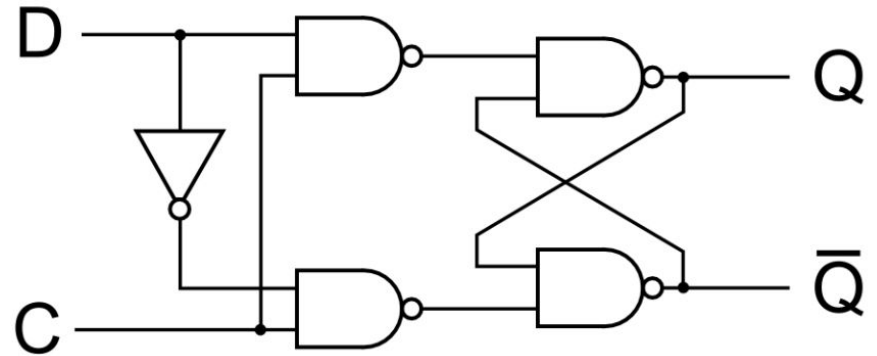
Almacenamiento de datos: Latch RS

S	R	Q(t+1)
0	0	-
0	1	0
1	0	1
1	1	Q(t)



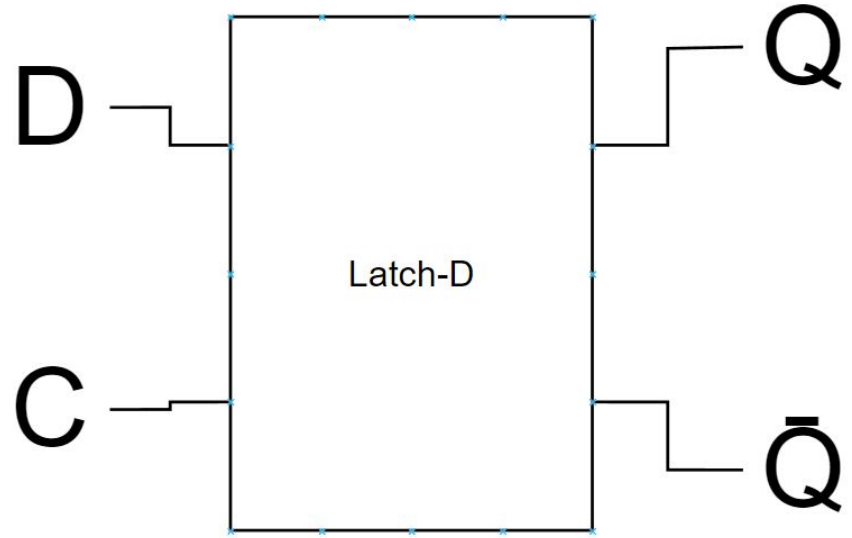
Almacenamiento de datos: Latch D

C	D	$Q(t+1)$
0	0	$Q(t)$
0	1	$Q(t)$
1	0	0
1	1	1



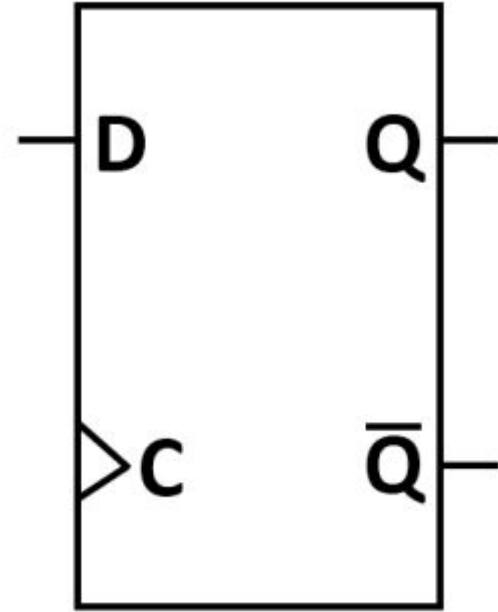
Almacenamiento de datos: Latch D

- D: Dato
- C: control
- Q: estado
- Esta es una abstracción usada para denotar el latch D



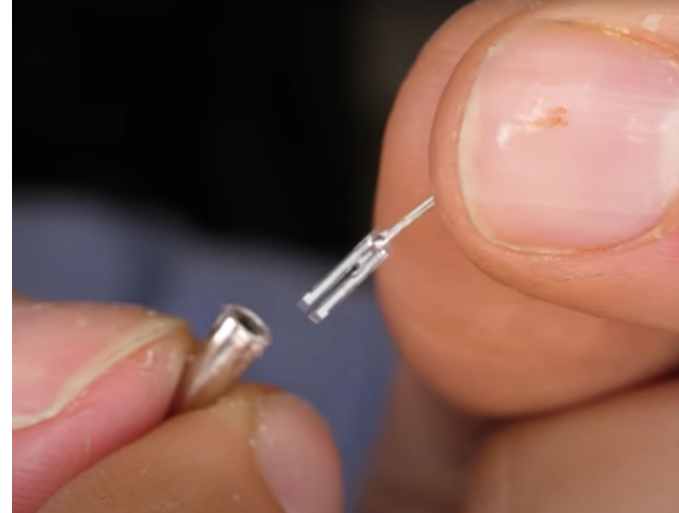
Almacenamiento de datos: Latch D

- Los **latches** se activan en un estado (1 ó 0), lo que no soluciona del todo nuestros problemas.
- Necesitamos una pieza similar, pero que se active sólo en **un instante dado**.

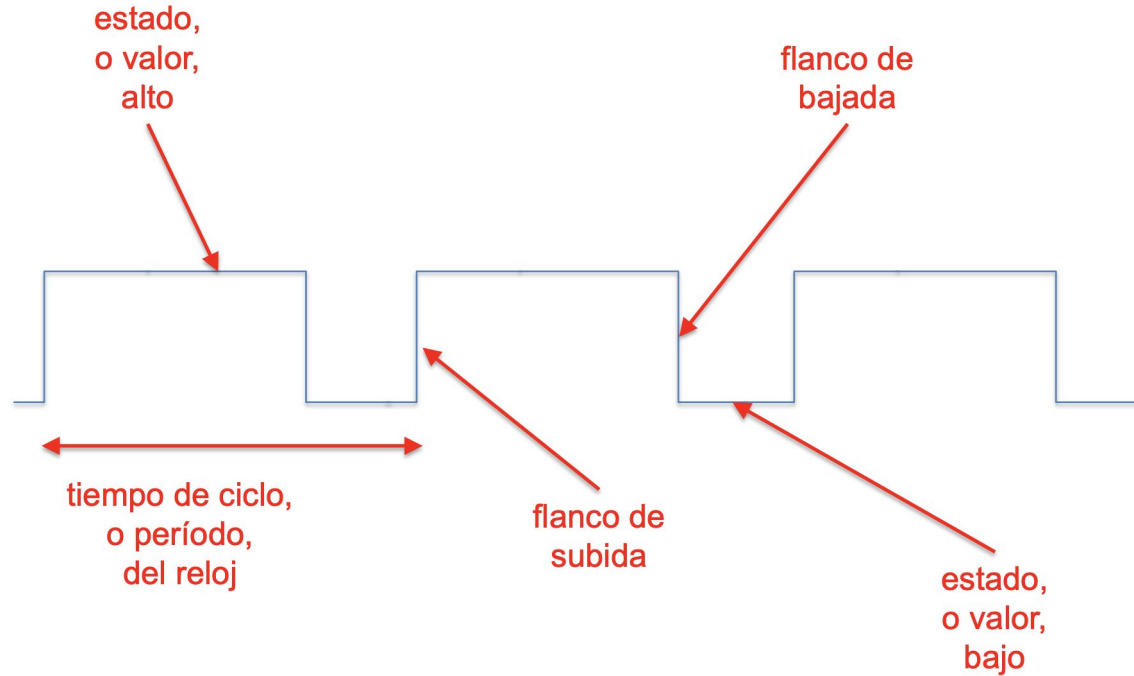


Almacenamiento de datos: Relojes

- Un reloj —en un circuito digital— es simplemente un circuito que emite una serie de pulsos, con dos propiedades:
- El ancho de pulso es preciso
- El intervalo entre pulsos consecutivos es preciso



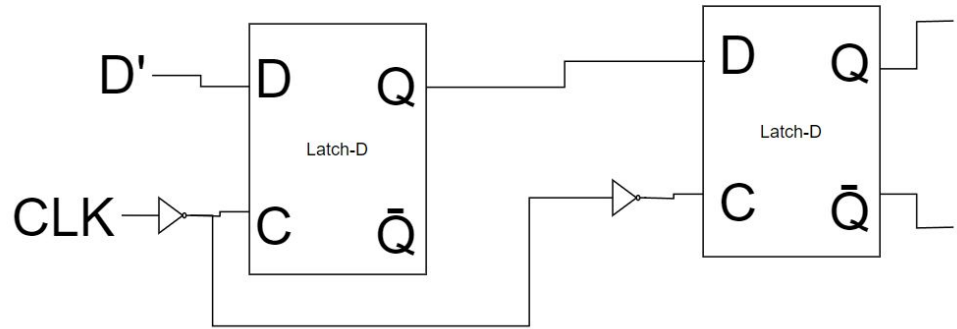
Almacenamiento de datos: Relojes



¿Dudas?

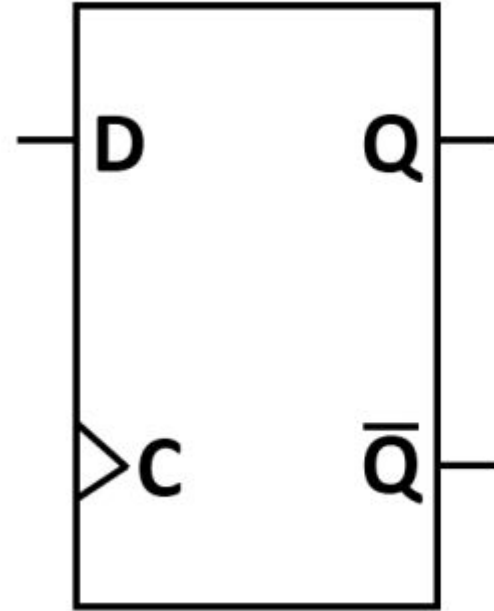
Almacenamiento de datos: Flip-Flop D

C	D	Q(t+1)
↑	0	0
↑	1	1
*	*	Q(t)



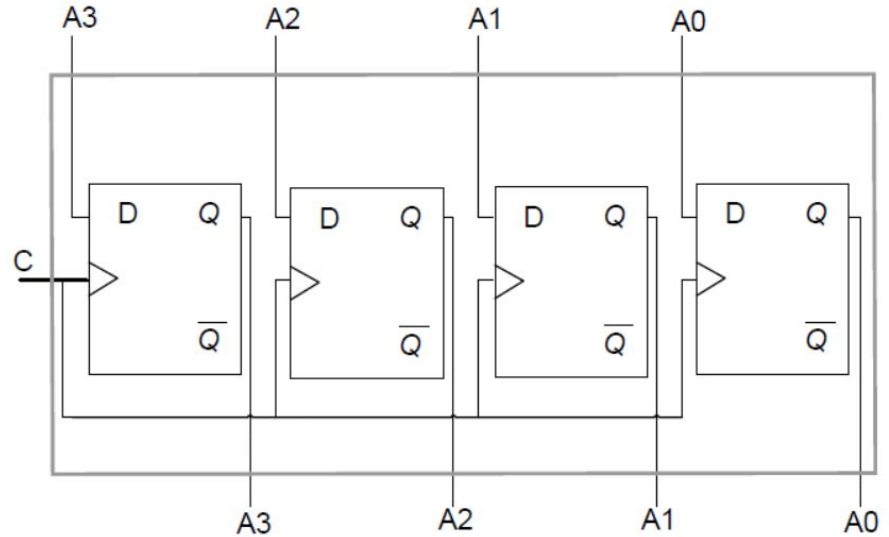
Almacenamiento de datos: Registro

- Los **flip-flop** se activan solo en una transición o flanco (subida o bajada).
- Cuando una componente tenga un input con símbolo un triángulo, significa que ese input es un clock.



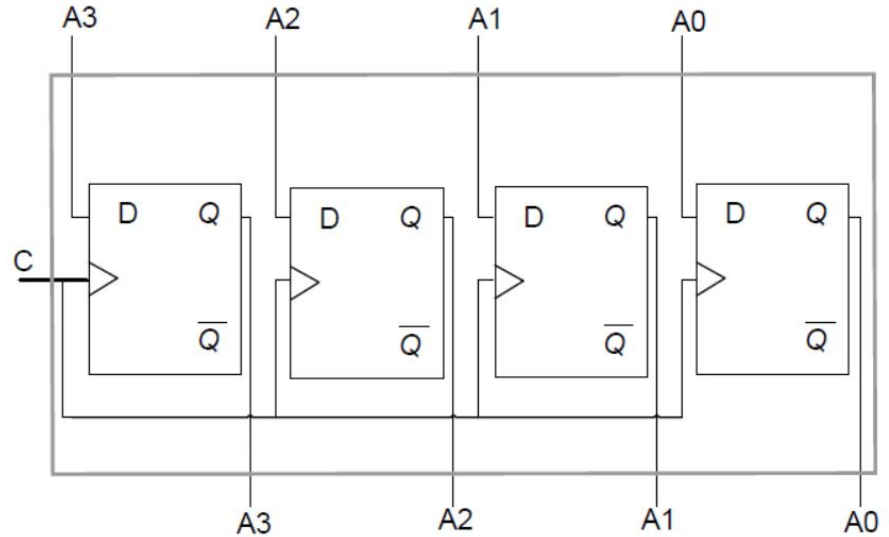
Almacenamiento de datos: Registros

- Llamaremos registros al conjunto de flip-flops
- Si queremos almacenar N bits necesitaremos N flip-flops



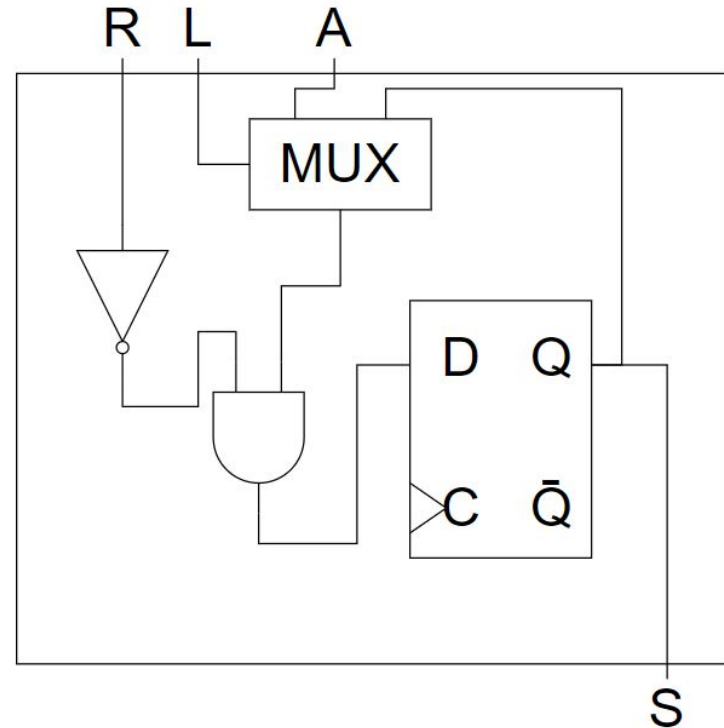
Almacenamiento de datos: Registros

- Llamaremos registros al conjunto de flip-flops
- Si queremos almacenar N bits necesitaremos N flip-flops
- Nos **falta control de almacenamiento**



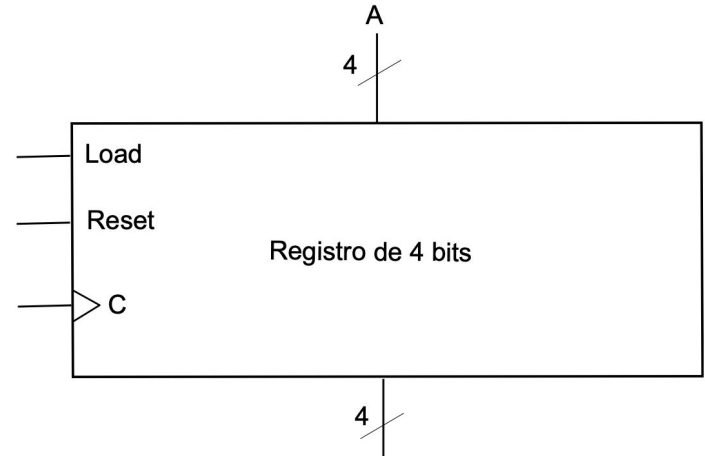
Almacenamiento de datos: Registros

C	L	R	S
↑	1	0	A
↑	*	1	0
*	0	0	Q(t)



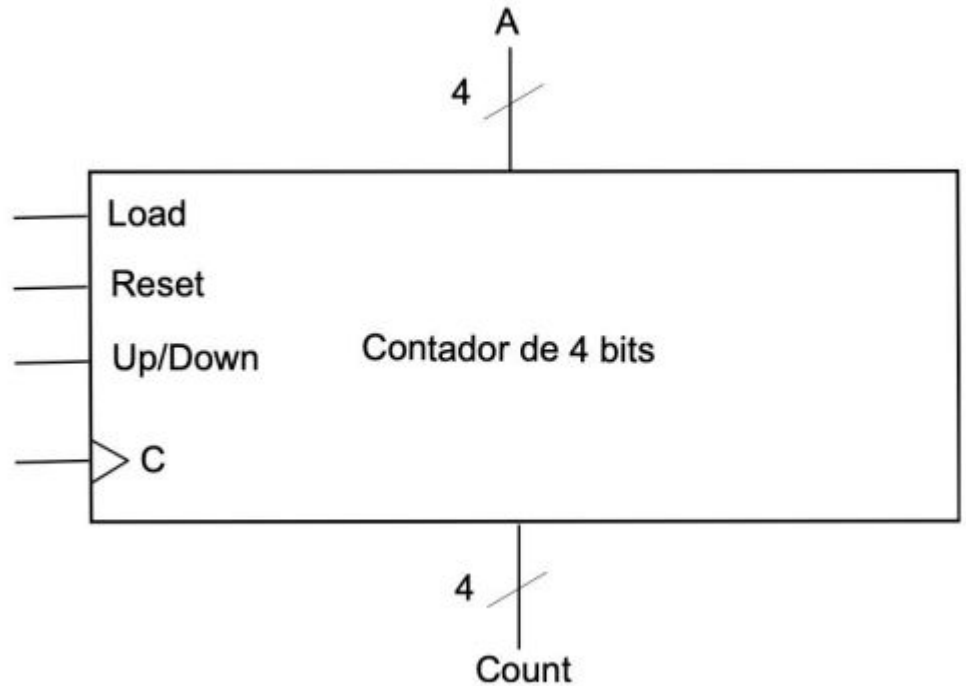
Almacenamiento de datos: Registros

- El registro tradicional tiene la función de carga y reset
- Se extiende con el manejo de varios flip-flops
- Nos permite almacenar unidades de información o **palabras**



Almacenamiento de datos: Registro Contador

- Con lo anterior se puede generalizar para tener un registro contador
- Una señal Up indicando que el valor almacenado se suma uno
- Una señal Down indicando que el valor almacenado se resta uno



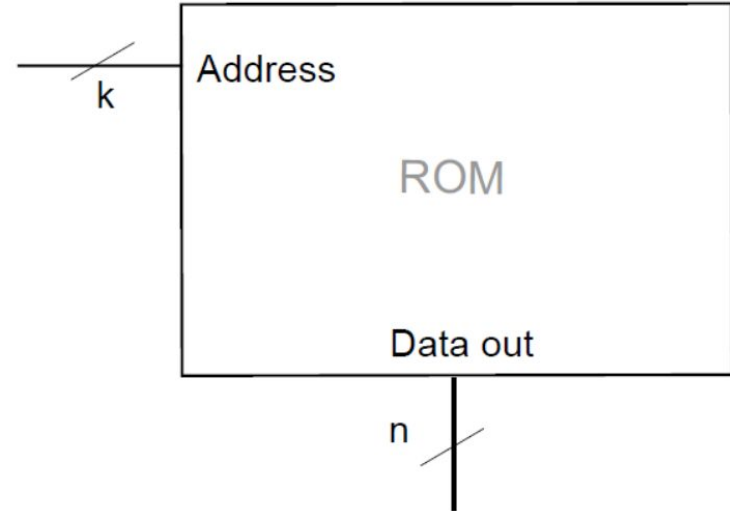
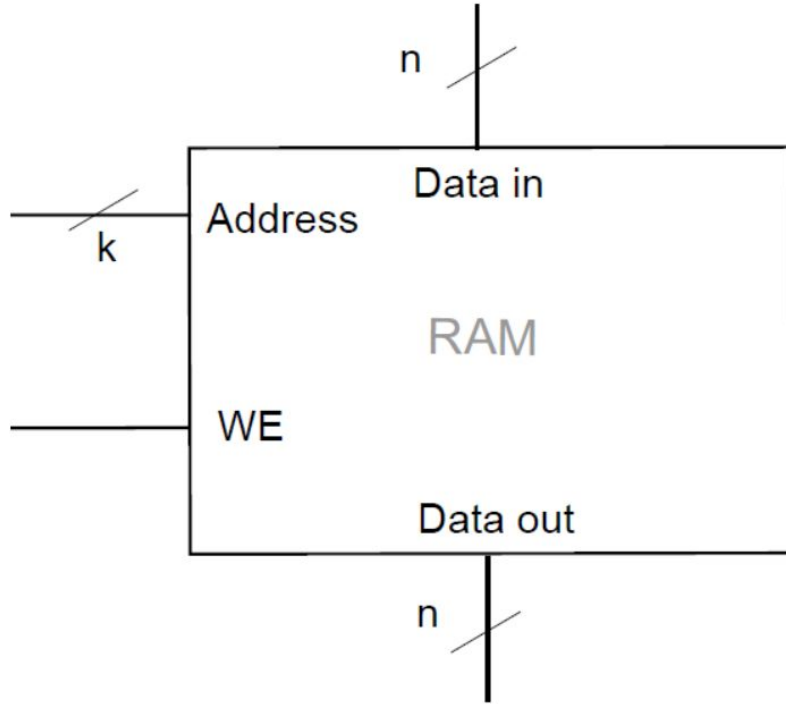
¿Dudas?

Almacenamiento de datos: Memorias

- Componentes que permite acceder, y en algunos casos, modificar distintos datos según una **dirección de memoria**
- El proceso de acceso se conoce como **direccionamiento**

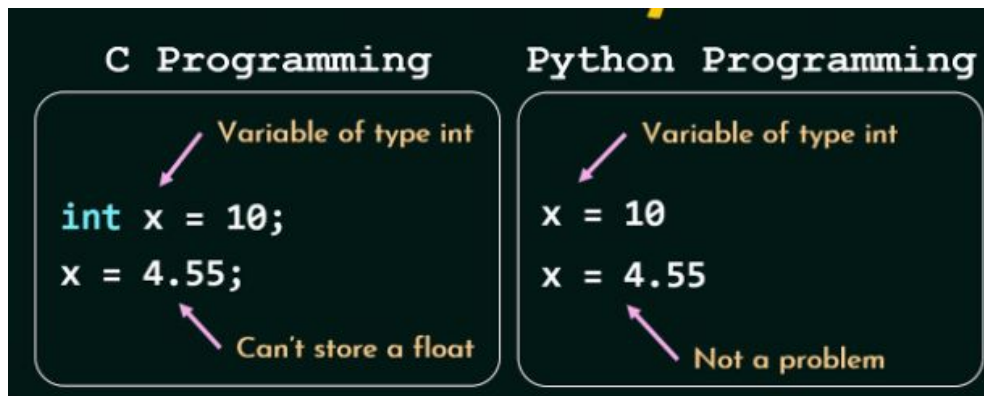
Dirección en decimal	Dirección en binario	Palabra
0	000	<i>Palabra₀</i>
1	001	<i>Palabra₁</i>
2	010	<i>Palabra₂</i>
3	011	<i>Palabra₃</i>
4	100	<i>Palabra₄</i>
5	101	<i>Palabra₅</i>
6	110	<i>Palabra₆</i>
7	111	<i>Palabra₇</i>

Almacenamiento de datos: Memorias



Almacenamiento de datos: Variable

- Valores temporales que se actualizan según flujo del código
- Se almacenan en distintos componentes, como puede ser registros o RAM
- Todo variable tiene un único **tipo de dato**



Almacenamiento de datos: Endianness

Address	Value
00	0x0F
01	0xA7
10	0x1D
11	0x23

- int 32 bits big endian:

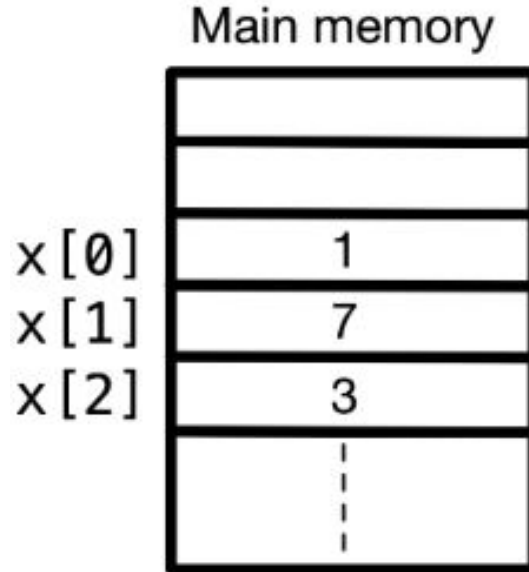
0x 0F A7 1D 23

- int 32 bits little endian:

0x 23 1D A7 0F

Almacenamiento de datos: Arreglos

- Es un caso especial de variable
- Se compone de cuatro atributos: **tipo de dato**, **endianness**, **largo** y **dirección de inicio**
- Se puede **indexar** cada elemento para acceder a ellos en nuestra memoria



¿Dudas?

Bibliografía

- Apuntes históricos. Hans Löbel, Alejandro Echeverría
- 02 - Operaciones Aritméticas y Lógicas
- 03 - Almacenamiento de datos
- D. Patterson, Computer Organization and Design RISC-V. Edition: The Hardware Software Interface. Morgan Kaufmann, 2020. Capítulos A.7-A.8-A.9. Página A-47, 792 en PDF

Bibliografía

- <https://github.com/IIC2343/Syllabus-antiores/tree/main/Enunciados>

The screenshot shows the GitHub interface for the repository `IIC2343 / Syllabus-antiores`. The left sidebar displays the file tree with the `Enunciados` directory selected. The main area shows the commit history for this directory, listing 17 commits with their names, messages, and dates.

Name	Last commit message	Last commit date
...		
Enunciados 2019-1	feat: add more solutions to previous exams; add previous auxiliary cl...	2 years ago
Enunciados 2019-2	feat: add more solutions to previous exams; add previous auxiliary cl...	2 years ago
Enunciados 2020-1	feat: add more solutions to previous exams; add previous auxiliary cl...	2 years ago
Enunciados 2021-1	add enunciados antiguos	2 years ago
Enunciados 2021-2	feat: add assessments for past semester	2 years ago
Enunciados 2022-1	feat: add assessments for past semester	2 years ago
Enunciados 2022-2	feat: add more solutions to previous exams; add previous auxiliary cl...	2 years ago
Enunciados 2023-1	feat: add assessments for past semester	2 years ago
Enunciados 2023-2	add missing test solutions for last semester	last year
Enunciados 2024-1	remove unused readme	10 months ago
Enunciados 2024-2	Delete Enunciados/Enunciados 2024-2/a	4 months ago
Compilado_IIC2343.pdf	Agregando compilado de Felipe	5 years ago

Introducción del curso:

- Un computador lo definimos como una **máquina programable que ejecuta programas.**
- Para programar necesitamos:
 - Datos: números (enteros, reales) , texto, imágenes, etc 
 - Operaciones: suma, resta, multiplicación, división, etc 
 - Variables: simples, arreglos 
 - Control de flujo: comparaciones, manejo de ciclos 
- Para poder tener esta última parte requerimos de comenzar el capítulo de **¡programabilidad!**

Clase 03 - Operaciones aritméticas y lógicas

Profesor: **IIC2343 - Arquitectura de Computadores**
- Felipe Valenzuela González
Correo:
frvalenzuela@alumni.uc.cl